

개발 흐름 총정리

Oct 3, 2026 · @Dave

0. 서비스 한 줄

CGM(리브레)을 쓰는 인슐린 사용자가 **사진 한 장과 인슐린 원탭**으로 기록하면, 혈당 곡선과 맞춰 **왜 그랬는지와 다음엔 어떻게 할지**를 일일·주간 리포트로 보여주는 iOS 앱.

1. 사용자 흐름 → 필요한 기능

단계	사용자 행동	구현 요소
시작	Apple 로그인, 인슐린 등록, 14일 PDF 공유	Sign in with Apple, 동의, 프리셋, 베이스라인 리포트
먹을 때	잠금화면 → 앱 카메라 → 확인	커스텀 카메라, 비전 AI, 개수 조정, 저혈당 15분 타이머
인슐린	위젯 버튼 탭	인터랙티브 위젯 + App Intents, App Group 저장
밤	알림 → 빠진 기록 확인	로컬 알림, 빈 시간대 질문
혈당	안내 3단계 → 리브레 공유 → 우리 앱	가이드 화면, Share Extension, PDF 파서
리포트	일일(사실) / 주간(판단)	에피소드 분석, 처치 판정, 그래프 3종, 교육 카드

2. 기술 스택

영역	선택
iOS	Swift, SwiftUI, iOS 17+, WidgetKit, App Intents, Share Extension, SwiftData(App Group), Swift Charts, AVFoundation
백엔드	Java 21, Spring Boot 3, Spring Security(JWT), PDFBox, Flyway, Testcontainers
DB	PostgreSQL
저장소	Cloudflare R2 또는 S3 (presigned URL)

영역	선택
AI	사진 인식: Claude Haiku 4.5 / 주간 문장화: Claude Sonnet 5.5 (Gemini Flash와 비교 테스트)
인프라	Lightsail 2GB + Docker Compose, GitHub Actions, 매일 pg_dump 백업

3. 데이터 흐름

[기록] 카메라 → R2 업로드 → API → job → Haiku → 검증 → food_catalog 계산 → 앱 폴링 위젯 → App Group 저장 → 앱이 서버로 동기화

[혈당] 리브레 PDF → Share Extension → API → PDFBox 곡선 추출 → glucose_reading upsert → 원본 삭제

[분석] 혈당 + 기록 시각 매칭 → episode(분류·분석 창·지표·판정)

[리포트] 혈당 들어온 날 → 일일(템플릿) / 7일 총족 → 주간(Sonnet 문장화 + 교육 카드)

4. DB 핵심 테이블

- 원본: users, insulin_preset, glucose_reading, intake_photo, insulin_event
- 기준: food_catalog (당류·처치 적합성), education_card (본문·출처), guide_content (안내 캡처)
- 분석 (재계산 가능): episode, daily_report, weekly_report
- 작업: job (종류·상태·재시도)

원칙: 원본은 건드리지 않고 분석은 언제든지 다시 계산, 시각은 timestampz, 건강정보는 암호화·동의·탈퇴 시 전체 삭제.

5. AI 원칙

- AI는 ***무엇이 몇 개***와 ***문장***만, 숫자·판정은 전부 코드
- API 키는 서버에만, 비동기 job, 스키마로 출력 강제 + 서버 검증
- FoodRecognizer / ReportWriter 인터페이스로 모델 교체 가능하게
- 예상 비용: 사용자당 월 500원 안팎

6. 개발 단계

단계	iOS	백엔드	완료 기준
0. 준비	프로젝트·타겟 3개·Core 패키지	Spring 골격, Docker Compose, Flyway, CI	로컬에서 앱↔서버 통신
1. 혈당	Apple 로그인	auth, PDF 파서 + 아버님 PDF 테스트	아버님 PDF가 DB에, CSV와 오차 검증 통과
2. 일일 그래프	Swift Charts 일일 화면	glucose 조회, 일일 리포트 기본	앱에 곡선이 보임
3. 사진	카메라, 업로드, 확인 화면	record, job 워커, food(Haiku), food_catalog	사진이 그래프 위에 붙음
4. 인슐린	위젯, App Intents, 동기화	인슐린 API, 이상치 경고	앱 안 열고 기록 → 서버 반영
5. 분석	처치 판정 화면	episode, 처치 판정	"초콜릿 → 반동" 자동 판정
6. 주간	주간 그래프 3종	좋은 날/아쉬운 날, 교육 카드, Sonnet	주간 리포트 완성 (데모 가능)
7. 실사용	Share Extension, 안내 화면, 온보딩, 알림	guide_content, 베이스라인	처음 설치부터 리포트까지 한 흐름
8. 출시 준비	접근성·다크모드·오류 화면	탈퇴·데이터 삭제, 모니터링, 백업	테스트플라이트 배포

1단계 PDF 파서를 먼저 끝내면 이후 모든 화면을 **아버님 실제 데이터**로 개발·검증할 수 있어요.

7. 인프라 준비

서버 한 대(Lightsail 2GB)에 Docker Compose로 Spring·PostgreSQL·HTTPS 프록시를 올리고, 사진과 백업은 R2에 뒀요. 예상 비용은 월 \$12~15예요.

순서	할 일	설명
1	계정·예산	AWS 계정 크레딧 확인, 서울 리전, 결제 알림(예: 월 \$20 초과 시) 설정
2	서버 생성	Lightsail 2GB Ubuntu, 고정 IP 연결, 방화벽은 22(내 IP 만)·80·443만 열기

순서	할 일	설명
3	서버 기본 설정	Docker·Docker Compose 설치, 스왑 2GB, 시간대 Asia/Seoul, 자동 보안 업데이트
4	도메인·HTTPS	도메인 A 레코드 → 서버 IP, Caddy 리버스 프록시로 인증서 자동 발급. iOS는 HTTPS만 허용(ATS)
5	Compose 구성	caddy, app (Spring), postgres (볼륨 마운트). 설정은 서버의 .env 에만
6	사진 저장소	R2 비공개 버킷 생성, presigned URL 발급용 액세스 키
7	CI/CD	GitHub Actions: 테스트 → Docker 이미지 빌드 → GHCR 푸시 → SSH로 서버에서 pull 후 재시작
8	백업	매일 pg_dump → R2 업로드(cron), 30일 보관, 복구 테스트 1회
9	모니터링	Spring Actuator /health 를 외부 헬스체크로 감시, 로그 로테이션, job 실패 시 디스코드 웹훅 알림
10	비밀값	Apple 키, AI API 키, DB 비밀번호, JWT 서명 키는 GitHub Secrets와 서버 .env 에만. 저장소에 커밋 금지

환경은 MVP에서 **로컬(Compose로 PostgreSQL만)**과 **운영** 두 개로 충분해요. 스테이징은 사용자가 생긴 뒤에 추가해요.

8. 백엔드 개발 흐름

백엔드는 **PDF 파서 → 기록 → 분석 → 리포트** 순서로, 각 모듈을 테스트와 함께 끝내고 다음으로 넘어가요.

패키지 구조

```

com.app
├─ auth      Apple 토큰 검증, JWT 발급·갱신, 탈퇴
├─ record    사진 기록, 인슐린 기록, 프리셋
├─ glucose   혈당 저장·조회, PDF 업로드 수신
├─ parser    PDFBox 곡선 추출
├─ food      비전 AI 호출, food_catalog 매칭
├─ analysis  에피소드 생성, 지표 계산, 처치 판정
├─ report    일일·주간 리포트, 교육 카드, LLM 문장화
├─ job       DB 작업 큐, 워커, 재시도
├─ content   안내 화면 캡처, 교육 카드 관리
└─ common    예외 처리, 응답 형식, 보안 설정
  
```

주요 API

메서드	경로	설명
POST	/auth/apple	Apple ID 토큰으로 로그인, JWT 발급
POST	/auth/refresh	토큰 갱신
DELETE	/me	탈퇴, 모든 데이터 삭제
GET / PUT	/insulin-presets	위젯 버튼 프리셋
POST	/intakes/upload-url	사진 업로드용 presigned URL 발급
POST	/intakes	사진 기록 생성 → 음식 인식 job 등록
GET / PATCH	/intakes/{id}	인식 결과 조회(폴링), 개수·분류·양 수정
POST	/insulin-events	인슐린 기록 (위젯 기록 일괄 동기화)
POST	/glucose/imports	리브레 PDF 업로드 → 파싱 job 등록
GET	/glucose/imports/{id}	파싱 진행 상태
GET	/glucose?from&to	기간별 혈당
GET	/reports/daily/{date}	일일 리포트
GET	/reports/weekly?start	주간 리포트
GET	/guide-content	리브레 안내 화면 이미지·문구

핵심 처리 흐름

PDF 가져오기

1. /glucose/imports 로 PDF 수신 → 임시 저장 → PDF_PARSE job 등록 → 즉시 202 응답
2. 워커가 일일 로그 페이지에서 목표 범위 박스와 00:00 위치로 축 보정, 곡선 좌표를 혈당값으로 변환
3. 보정 실패 시 "지원하지 않는 형식"으로 실패 처리하고 알림
4. (user_id, measured_at) 기준 upsert → 원본 PDF 삭제
5. 영향받은 날짜에 ANALYZE job 등록

사진 인식

1. 앱이 presigned URL로 R2에 업로드 → /intakes 호출 → FOOD_RECOGNIZE job 등록
2. 워커가 이미지를 줄여 Haiku 호출, tool use 스키마로 JSON 수신
3. 서버 검증 실패 시 1회 재시도, 그래도 실패면 NEEDS_REVIEW
4. food_catalog 매칭으로 당류·처치 적합성 계산 → DONE

분석·리포트

1. **ANALYZE** job이 그날 혈당과 기록을 시각으로 매칭
2. 60~90분 안의 섭취는 한 에피소드로 묶고, 분류별 분석 창 적용 (식사 3~5시간, 처치 2시간, 술 12시간)
3. 저혈당 처치는 직전 혈당 70 미만이면 자동 분류, 회복 시간·반동 여부 판정
4. 일일 리포트는 템플릿으로 즉시 생성
5. 7일이 채워지면 **WEEKLY_REPORT** job → 좋은 날/아쉬운 날 분류, 교육 카드 매칭, Sonnet 문장화

테스트와 브랜치

- **PDF 파서**: 아버님 PDF를 픽스처로 두고 CSV 값과 평균 오차 1mg/dL 이내인지 검증
- **분석 로직**: 혈당·기록 샘플을 넣고 판정 결과를 확인하는 단위 테스트 (AI 호출은 모킹)
- **통합 테스트**: Testcontainers로 실제 PostgreSQL에서 API 흐름 확인
- **브랜치**: **main** 보호, 기능별 브랜치 → PR → CI 통과 후 머지 → 자동 배포

9. 착수 전에 확인할 것

1. **리브레 앱 실제 화면 캡처**: 메뉴 → 보고서 → 공유 경로, PDF 공유 시 우리 앱이 공유 시트에 뜨는지
2. **PDF 형식 다양성**: 기간을 7일·14일·30일로 바꿔도 일일 로그 페이지 구조가 같은지
3. **비전 AI 비교**: 테스트 사진 50장(한식, 간식, 저혈당 처치 음식, 포장 제품)으로 Haiku vs Gemini Flash
4. **교육 카드 출처**: 대한당뇨병학회 진료지침 원문 확보, 가능하면 의료진 검토
5. **개인정보**: 건강정보 동의서, 외부 AI 전송 고지, 리브레 캡처의 상표 사용 검토
6. **앱 이름**